

ORM (Object Relational Mapping)

→ It's a technique that allows us to interact with relational DB using objs instead of writing sql queries manually.

- Instead of writing,
 - select * from student;we do,
 - studentRepo.findAll(),

→ Without ORM we need to write manual JDBC programming which takes a lot of time

And we face problems like:

- Too much boilerplate code
- Need SQL everywhere
- Hard to maintain
- Error prone

→ ORM performs mapping
It maps,

Java object



Database Table

ex., `Student S = new Student(1);`

This becomes Database
(Student table)

also, `S.setName("Rushabh");`

becomes

`Insert into Student (name)`

`Values ('Rushabh')`

Meaning of Mapping

mapping simply means
connecting two things together

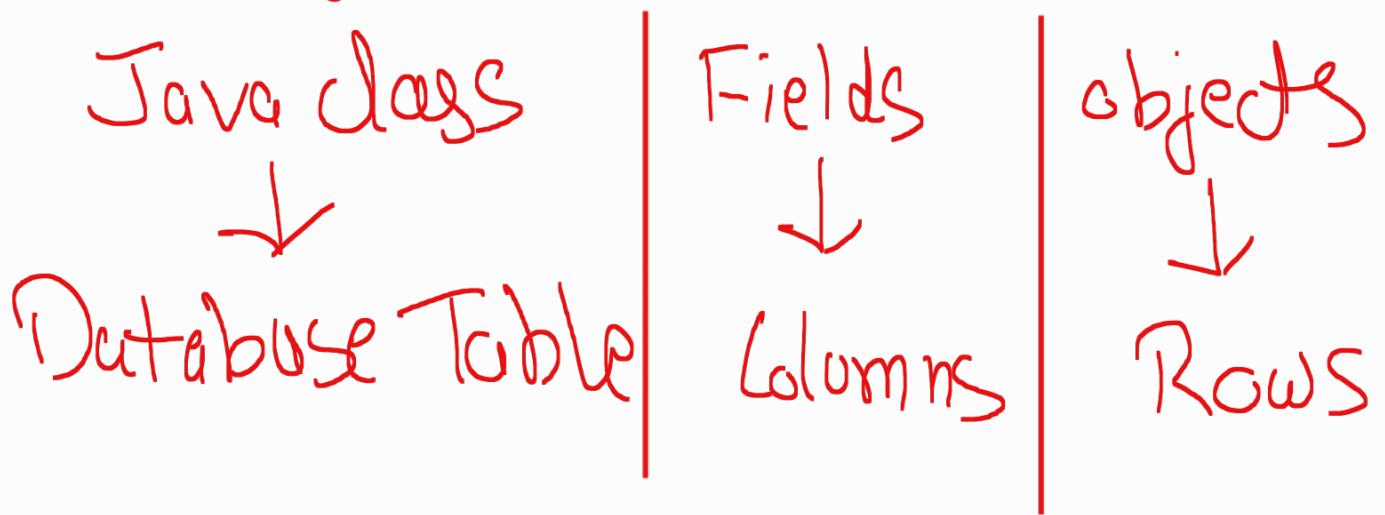
ex., The table

```
Public class Student {  
    int id;  
    String Name;  
}
```

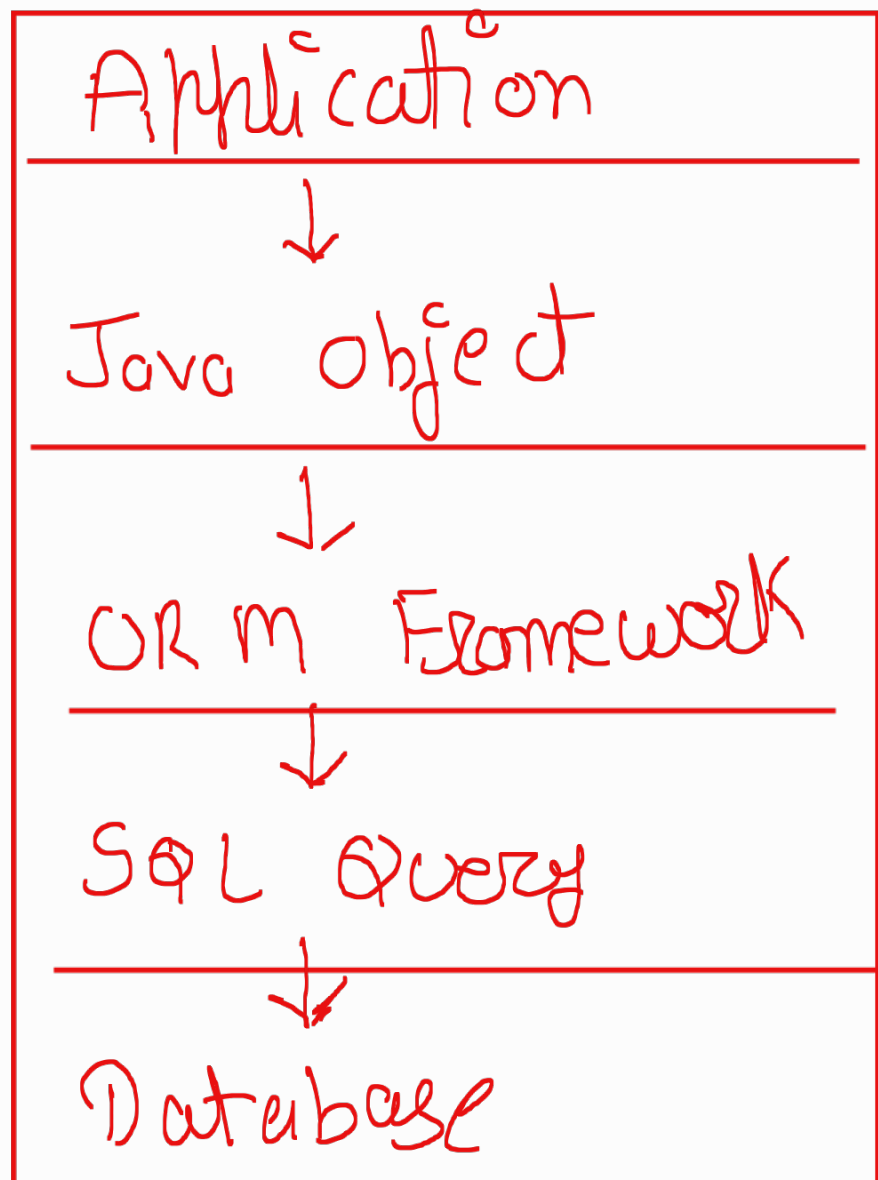
is mapped to Database
Student

The variables id, name
maps to id (column), name (column)
resp

These are ,



Architecture of ORM

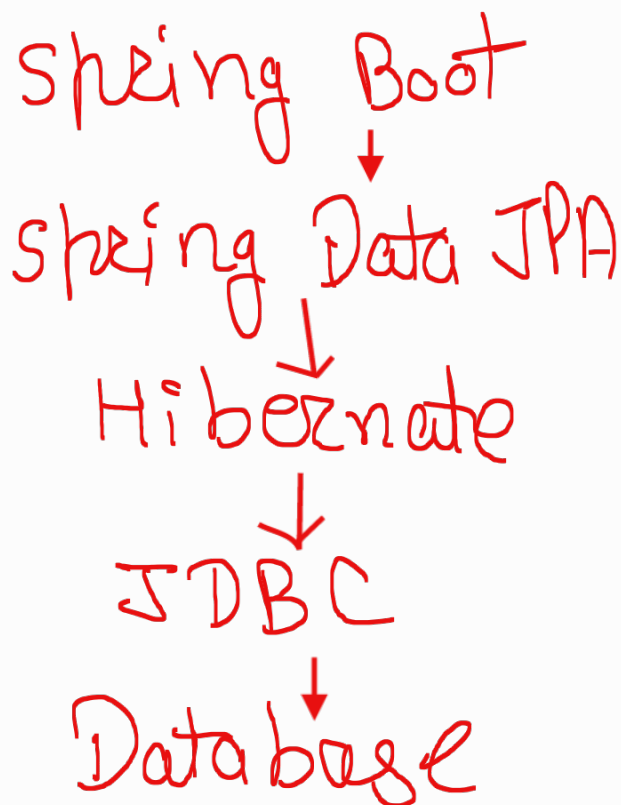


Popular ORM frameworks are:

- Hibernate
- JPA (specification)
- EclipseLink
- OpenJPA

ORM in Java

Java mainly uses

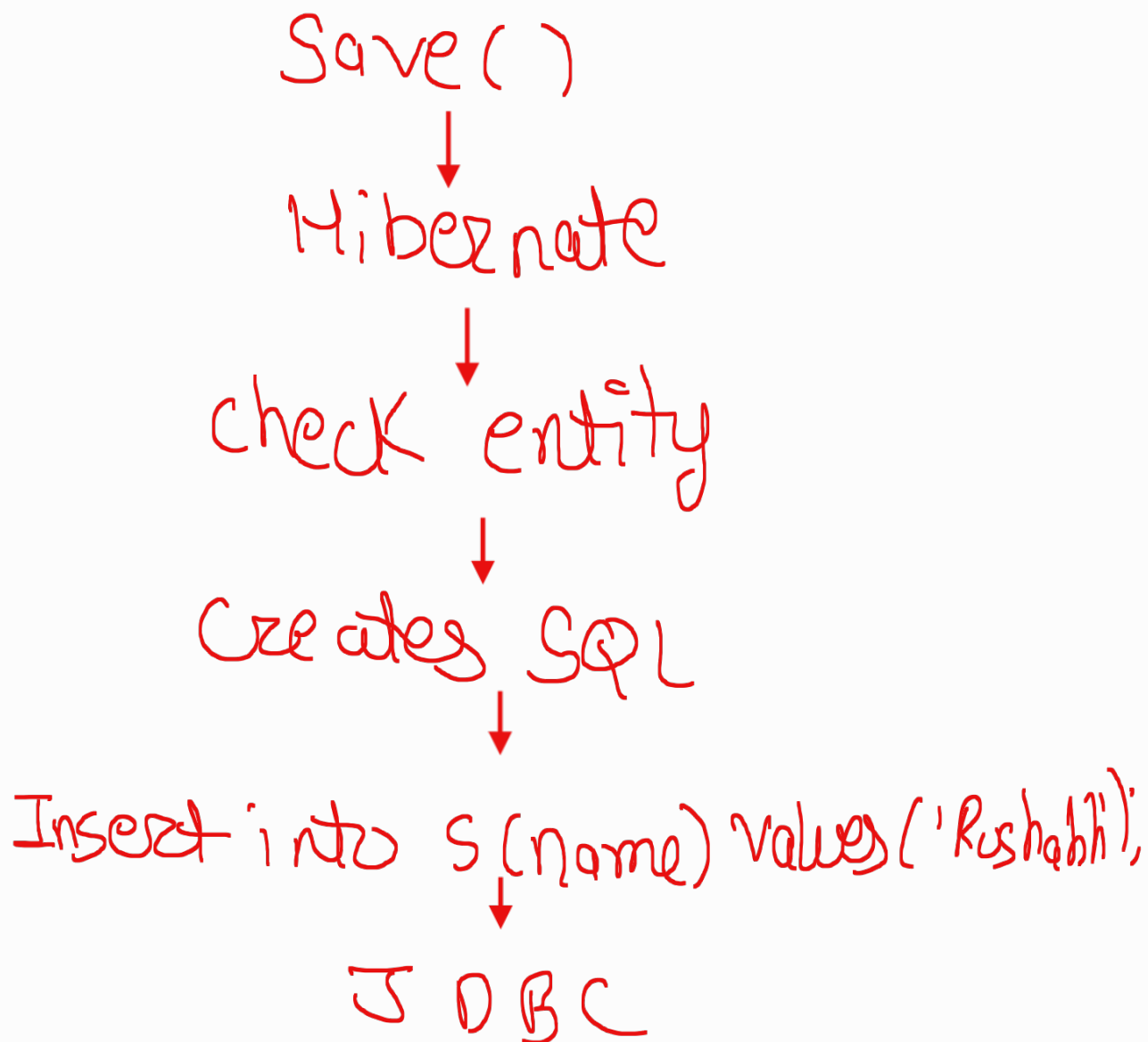


Working on ORM internally

suppose,

```
student s = new Student();  
s.setName ("Rushabh");  
repo.save (s);
```

internally,



↓ Database

Level operations using ORM

① Create

```
repo.save(student);
```

② Read

```
repo.findById(1);
```

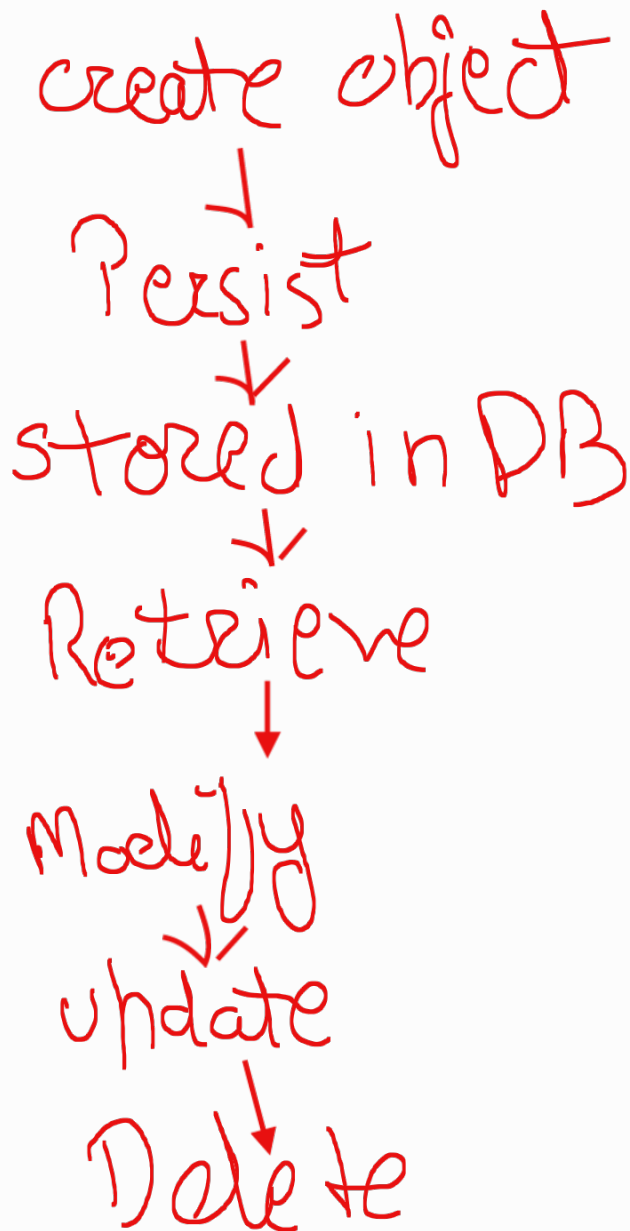
③ Update

```
student.setName("Amit")  
repo.save(student);
```

④ Delete

```
repo.delete(student)
```

ORM lifecycle



Advantages of ORM

- Less code
- Database Independent
- Easier maintenance
- Automatic mapping

→ Object Oriented Programming